

AI-Based Food Recognition With Nutrition-Aware Recommendations

Jaspreet Singh and Inderpal Singh

Abstract

This article describes an educational Indian-food recognition application and a reproducibility audit of its evidence. The application joins YOLO11s detection, database nutrition per 100 g, user-estimated gram portions, an explicitly saved browser-only meal journal, and context-labeled recommendations. The existing export contains 48,693 images across 72 classes. A full read-only audit found 138 exact-duplicate groups spanning the original partitions, affecting 276 images, and strict annotation issues in 65 images. Consequently, the previously reported test mAP@50 of 0.820 and mAP@50:95 of 0.680 are historical observations, not estimates of clean independent generalization. Nutrition lookup is available for 19/72 classes with normalized exact matching, 55/72 with automatic matching without precomputed mappings, and 72/72 with the current mapping layer; these counts do not measure nutritional correctness. The work supplies fingerprinted audit artifacts, a source-family-grouped candidate split with quarantine, and an independent mapping-review protocol. It demonstrates an inspectable application and identifies necessary validation work, without claiming a novel detector, measured food mass, or medical efficacy.

Index Terms

Food recognition, object detection, Indian food dataset, nutrition databases, personalized recommendations, dietary assessment.

Jaspreet Singh and Inderpal Singh are with the Department of Computer Science and Engineering, National Institute of Technology Srinagar, Kashmir 190006, India. Email: [student email].

This work was developed as part of a final year project on AI-based food recognition with nutrition-aware recommendations.

SUBMISSION-BLOCKING AUDIT: SEPTEMBER 2026

The five source versions have now been recovered from local export metadata [1], [2], [3], [4], [5]; this does not establish blanket image-level redistribution permission. A read-only audit of all 48,693 exported images identified 138 groups of byte-identical and decoded-pixel-identical images spanning splits: 63 train–validation, 56 train–test and 19 validation–test groups, affecting 276 images. Strict annotation checks flagged 65 images, including zero-width boxes. Archived detector results below are historical observations, not proof of clean held-out generalization. Original-source and augmentation-family grouping, annotation review, and new independent evaluation are required before external submission. No dataset was edited and no detector was retrained during this audit.

The measured lookup-availability ablation covers 19/72 classes with normalized exact matching, 55/72 with automatic lookup without precomputed mappings, and 72/72 with the current mapping layer. This is not mapping accuracy: current mappings are not independent ground truth, and 21 have heuristic scores below 0.9. A blank reviewer template and scorer accompany `research/evidence/audit.json`. The workbook is byte-identical to the authors’ pinned public artifact; redistribution terms, author details and a venue still require confirmation. Historical “VERIFIED” labels do not establish nutritional or clinical equivalence.

The current application uses gram-based portions and explicitly saved meals, not automatic consumption from scans. Health selections are session-only. Groq receives included food names and database nutrition per 100 g, goal and optional health context, but not photos, selected gram amounts or journal history. Existing clinical rules remain unvalidated, and health badges only identify response context. Historical screenshots should not be read as the September interface.

A source-family recovery pass then indexed 40,797 source images and joined exact identities with within-export Roboflow filename families. It found 5,548 candidate families spanning old partitions. A deterministic review-only assignment places 26,384 images in train, 5,655 in validation and 5,652 in test, with all 72 classes represented in each. Another 11,002 images are quarantined by whole group for unresolved lineage or annotation review, not deleted. Known group overlap is zero in the candidate partitions, but upstream augmentation, within-group repeats and unrecognized cross-source near duplicates remain limitations. This is preparation for review,

not a new benchmark result; the existing detector was not retrained.

I. INTRODUCTION

Manual food logging is difficult to sustain because it requires repeated identification, quantity estimation, database search, and entry correction. Image-based food recognition can reduce that effort by allowing a user to photograph a meal and receive detected food items. However, a visual label alone is not sufficient for dietary feedback. A user does not only need to know that an image contains `samosa`, `dal`, or `palak_paneer`; the label must be connected to calories, protein, carbohydrates, fat, and practical guidance.

Indian meals make this problem harder than ordinary object detection. A single plate may contain rice, curry, dal, chutney, fried bread, sweet dishes, and beverages. Dishes have regional variants and inconsistent names. The same food may appear as `WhiteRice`, `satham`, or `rice`; chutney labels may appear as `coconut_chutney`, `thengai_chutney`, or source-specific spellings; and visually similar dishes such as biryani and pulao may differ nutritionally by preparation and ingredients. This makes a direct detector-to-string-match approach fragile.

The core claim of this work is that nutrition-aware food recognition is a system problem rather than only a model problem. Object detection provides candidate labels and bounding boxes. A nutrition layer must then decide what each label means in a food-composition database. If that mapping is weak, the application can return precise-looking but wrong macro values. For example, `aloo_gobi` should resolve to potato cauliflower, and `palak_paneer` should resolve to spinach paneer. A fuzzy string match to a visually or linguistically adjacent but nutritionally different dish is not acceptable for a health-oriented interface.

This paper presents a working system and reports both model-level and pipeline-level validation. The contributions are:

- a dataset engineering pipeline that merges five YOLO-format Indian food sources into a 72-class canonical dataset;
- historical YOLO11s results, qualified by a reproducible audit of partition overlap;
- a conservative class-to-database mapping strategy for Indian nutrition lookup;

- a deterministic macro-target engine that adapts carbohydrate, protein, and fat targets to dietary goals and health profiles;
- a full-stack prototype that links food detection, nutrition retrieval, serving controls, daily macro tracking, and recommendation generation.

Unlike work that treats food recognition as a single-image classification task, this system is designed for multi-item meal images and user-facing nutrition feedback. Unlike direct vision-language nutrient estimation, the system does not ask a language model to invent calories or macro values. Nutrition values come from a structured database, while generated recommendations use already-computed backend values, user goals, and selected health conditions as input context.

The remainder of this article is organized as follows. Section II formalizes the problem. Section III reviews related work. Section IV describes the system architecture. Sections V and VI present the dataset pipeline and detector evaluation. Section VII explains nutrition database construction and mapping. The remaining sections describe the prototype, validation outcomes, ablation evidence, discussion, practical considerations, limitations, and conclusion.

II. PROBLEM FORMULATION

A. Notation and Objective

Let $\mathcal{X}$ denote the space of meal images and let $\mathcal{C} = \{c_1, c_2, \dots, c_K\}$ denote the canonical food-label vocabulary used by the detector. In this work $K = 72$. A source dataset s provides images $\mathcal{X}^{(s)}$ and annotations $\mathcal{A}^{(s)}$, where an annotation for image x_i is a set

$$A_i^{(s)} = \{(y_{ij}^{(s)}, \mathbf{b}_{ij}^{(s)})\}_{j=1}^{n_i}. \quad (1)$$

Here $y_{ij}^{(s)}$ is a source-specific class name and $\mathbf{b}_{ij}^{(s)} = (x_{ij}, y_{ij}, w_{ij}, h_{ij})$ is a normalized bounding box. The dataset-engineering task is to learn or define a deterministic canonicalization function

$$\phi_s : \mathcal{Y}^{(s)} \rightarrow \mathcal{C}, \quad (2)$$

where $\mathcal{Y}^{(s)}$ is the raw label vocabulary for source s . The merged training corpus is therefore

$$\mathcal{D} = \bigcup_s \{(x_i, \{(\phi_s(y_{ij}^{(s)}), \mathbf{b}_{ij}^{(s)})\}_{j=1}^{n_i})\}. \quad (3)$$

The detector is a parameterized function $f_\theta : \mathcal{X} \rightarrow \mathcal{P}$ that maps an image to a set of predictions

$$f_\theta(x) = \{(\hat{c}_q, \hat{\mathbf{b}}_q, \hat{p}_q)\}_{q=1}^{Q_x}, \quad (4)$$

where $\hat{c}_q \in \mathcal{C}$ is the predicted food class, $\hat{\mathbf{b}}_q$ is the predicted bounding box, $\hat{p}_q \in [0, 1]$ is the confidence score, and Q_x is the number of retained detections after non-maximum suppression.

Let the nutrition database be

$$\mathcal{N} = \{(n_\ell, \mathbf{m}_\ell, u_\ell, z_\ell)\}_{\ell=1}^L, \quad (5)$$

where n_ℓ is a food name, $\mathbf{m}_\ell = [e_\ell, p_\ell, h_\ell, g_\ell]^\top$ is a nutrition vector containing energy, protein, carbohydrate, and fat, u_ℓ is the serving basis, and z_ℓ is provenance metadata. The system requires a mapping operator

$$M : \mathcal{C} \rightarrow \mathcal{N} \cup \{\emptyset\} \quad (6)$$

that maps each detector class to a nutrition row or returns $\emptyset$ when no accepted row exists.

For a detected meal image, the estimated nutrition vector is

$$\hat{\mathbf{m}}(x, \mathbf{s}) = \sum_{q=1}^{Q_x} \mathbb{I}[M(\hat{c}_q) \neq \emptyset] s_q \mathbf{m}_{M(\hat{c}_q)}, \quad (7)$$

Here $s_q = w_q/100$ for an included detection with estimated weight w_q grams, and $s_q = 0$ for an excluded detection. The interface offers 25–1,000 g in 25 g steps, defaulting to 100 g. Equation (7) is a known subtotal: any included unmapped item marks it incomplete, never a zero-nutrient item. The journal records consumption only when the user explicitly saves a meal.

B. System Risk Decomposition

The end-to-end error in a nutrition-aware food-recognition system is not only detector error. It can be decomposed into at least four components:

$$\mathcal{E}_{sys} = \mathcal{E}_{det} \cup \mathcal{E}_{map} \cup \mathcal{E}_{portion} \cup \mathcal{E}_{rec}, \quad (8)$$

where $\mathcal{E}_{det}$ is detection error, $\mathcal{E}_{map}$ is class-to-database mapping error, $\mathcal{E}_{portion}$ is serving or portion-size error, and $\mathcal{E}_{rec}$ is recommendation-generation error. A detector with high mAP can still produce poor dietary feedback if $\mathcal{E}_{map}$ is large. For example, a correct detection of palak_paneer still fails nutritionally if it maps to a generic paneer curry with substantially different spinach, oil, or cream composition.

This paper therefore defines the main objective as a constrained system problem:

$$\max_{\theta, M} \text{mAP}(f_{\theta}; \mathcal{D}_{test}) + \lambda \text{Cov}(M) \quad \text{s.t.} \quad \text{Unsafe}(M) = 0, \quad (9)$$

Here $\text{Cov}(M)$ is lookup availability, not accuracy. Equation (9) expresses a desired design constraint, not an optimized objective or achieved safety property. The scalar λ is not fitted. No independent reference judgments demonstrate $\text{Unsafe}(M) = 0$; mapping and advice quality remain unmeasured.

C. Research Questions

The review manuscript is organized around four research questions:

- RQ1: Can heterogeneous YOLO-format Indian food datasets be normalized into a single canonical label space without losing class coverage?
- RQ2: What can the archived detector results establish given measured partition overlap?
- RQ3: Does conservative class-to-database mapping provide complete nutrition coverage while avoiding unsafe fuzzy substitutions?
- RQ4: Does the full-stack prototype preserve auditability from image upload through detection, mapping, macro calculation, and recommendation generation?

D. Matrix View of Mapping and Nutrition Aggregation

The class-to-nutrition mapping can also be represented as a sparse assignment matrix

$$\mathbf{A} \in \{0, 1\}^{K \times L}, \quad (10)$$

where

$$A_{k\ell} = \begin{cases} 1, & \text{if canonical class } c_k \text{ maps to nutrition row } n_\ell, \\ 0, & \text{otherwise.} \end{cases} \quad (11)$$

For an at-most-one-row lookup policy, each row of $\mathbf{A}$ has at most one nonzero entry:

$$\sum_{\ell=1}^L A_{k\ell} \leq 1, \quad \forall k \in \{1, \dots, K\}. \quad (12)$$

Let $\mathbf{M}_N \in \mathbb{R}^{L \times 4}$ be the nutrition matrix where row ℓ is the macro vector for database row n_ℓ .

Let $\mathbf{z} \in \mathbb{R}^K$ be the detected class count or serving-weighted class vector for an image. The meal macro vector can then be written compactly as

$$\hat{\mathbf{m}} = \mathbf{z}^\top \mathbf{A} \mathbf{M}_N. \quad (13)$$

Equation (13) shows why the mapping matrix is central. Even if $\mathbf{z}$ is correct, incorrect nonzero entries in $\mathbf{A}$ produce incorrect macro totals.

III. RELATED WORK

A. Food Computing and Image-Based Dietary Assessment

Food computing connects image analysis, recipe understanding, nutrition estimation, and dietary behavior [6]. Smartphone systems such as FoodCam demonstrated real-time mobile food recognition for dietary logging [7]. Im2Calories framed the broader challenge of estimating food energy from meal images and showed that recognition alone is not enough when portion size and nutrient composition are uncertain [8]. These works motivate the present system, but this paper focuses on regional Indian foods and the connection between detector labels and nutrition database rows.

B. Detection Models for Multi-Item Meals

Object detection is suitable for meal images because multiple food items can appear in the same photograph. Two-stage detectors such as Faster R-CNN separate region proposal and classification [9]. YOLO-style one-stage detectors directly predict boxes and class probabilities, making them appropriate for interactive applications [10], [11]. Recent Indian-food work has also moved toward multi-item detection and diet-logging workflows: IndianFoodNet compared YOLO-family detectors for Indian food items [12], while dish-detection work on Indian platters reported large manually annotated platter datasets and evaluated several detection architectures for automated logging [13]. The implementation in this work uses Ultralytics YOLO11 tooling because it supports current YOLO training workflows, standard YOLO annotations, exported metrics, and Python integration [14].

C. Dataset Bias and Regional Food Naming

Datasets remain a central challenge. General object detection benchmarks such as COCO provide standardized object categories and annotations [15]. Regional food datasets are less uniform. They often use source-specific class names, inconsistent spelling, overlapping labels, and uneven annotation quality. Recent Indian cuisine benchmarks such as Khana show the scale and diversity of Indian food-image data, with classification, segmentation, and retrieval baselines over a broad Indian taxonomy [16]. Thali-oriented research further shows why Indian meals need multi-component analysis rather than a single dominant food label [17].

For nutrition-aware recognition, there is also a second mismatch: visual class names are short machine-learning labels, while food-composition databases contain descriptive recipe names, preparation details, and source metadata.

For Indian food recognition this mismatch is especially important because regional names often refer to the same preparation family. A rice dish can appear as `satham`, `WhiteRice`, or `rice`. A lentil dish can appear as `dal`, `daal`, `varan`, or a regional `sambar`-like preparation. A direct source-label union would therefore inflate the class count while fragmenting examples for the same dish. Conversely, aggressive merging can hide real nutritional differences, such as `chicken_biryani`, `mutton_biryani`, and `veg_briyani`. The present work treats label

normalization as a controlled data-engineering step rather than as a purely automatic text-cleaning operation.

D. Nutrition Databases and Recommendation Interfaces

Nutrition lookup must therefore be treated as a first-class system component. Structured databases are preferable for calories and macro values because direct language-model nutrient estimates can appear fluent while being numerically unreliable. Recent work on Indian food-composition data and FKG.in emphasizes the need for structured Indian nutrition resources and explicit food-composition reasoning [18], [19]. At the same time, evaluations of LLM-based nutrient estimation from meal photographs report that visual identification and exact nutrient estimation remain different problems [20]. This work therefore uses a local SQLite database derived primarily from Indian food-composition data, with source-backed supplemental rows only where the available database sheet lacks safe entries.

The application separates estimation from advice. The detector estimates food classes and boxes. SQLite supplies per-100-g nutrient values. The frontend scales known values by included gram portions and sums explicitly saved meals for daily totals; goals do not alter a food’s database values. A separate endpoint calculates goal targets from existing policy. Recommendations receive food names, per-100-g values, the goal and session-only health context, not portion weights, calorie targets or journal history. These boundaries allow separate software tests but do not establish nutritional correctness.

IV. SYSTEM ARCHITECTURE

The system follows a layered architecture with four main layers: dataset and nutrition data, AI inference, backend orchestration, and frontend interaction. Fig. 1 summarizes the pipeline. Source datasets are normalized into a canonical YOLO dataset. A YOLO11s model detects visible food objects. The backend maps detected class labels to nutrition database entries, computes macro targets from user context, and returns structured data for the frontend and recommendation service.

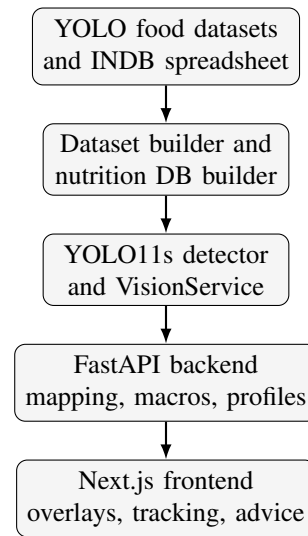


Fig. 1. System architecture for nutrition-aware Indian food recognition.

A. Design Requirements

The architecture is guided by six requirements. First, the detector must support multiple items per image because Indian meals commonly contain several visible dishes. Second, the nutrition lookup must be auditable; each macro value should be traceable to a database row and source. Third, the system must avoid unsafe automatic substitutions. Fourth, the application must expose enough information for user correction, especially serving-size adjustment. Fifth, macro targets must respond to dietary goals and health profiles. Sixth, the recommendation layer must not become the source of numeric nutrition values.

TABLE I
SYSTEM REQUIREMENTS FOR REVIEWER-FACING EVALUATION.

Requirement	Implementation choice	Reviewer-visible evidence
Multi-item recognition	YOLO object detector with bounding boxes	validation and test mAP, prediction grids
Label consistency	canonical 72-class label space	normalization examples and class counts
Nutrition traceability	SQLite rows with source meta-data	mapping and supplemental-row tables
Unsafe-match control	conservative mapping order with empty fallback	mapping equation and lookup tests
User correction	gram-based portions and macro totals	frontend/API contract and prototype figure
Profile-aware guidance	goal and health-condition macro modifiers	macro equation, profile table, and recommendation contract

The backend exposes endpoints for health checks, image analysis, nutrition lookup validation, macro calculation, dietary goals, health-condition options, and recommendations. The image analysis endpoint validates uploaded files, runs inference, attaches nutrition records, and returns labels, confidence scores, bounding boxes, image dimensions, macro values, and source fields.

B. Inference Data Contract

The image-analysis endpoint returns a structured list of detections. For detection q , the backend response can be represented as

$$d_q = (\hat{c}_q, \hat{p}_q, \hat{\mathbf{b}}_q, \eta_q, \mathbf{m}_q, z_q), \quad (14)$$

Here η_q is the display name, $\mathbf{m}_q$ contains database values per 100 g, and z_q contains source metadata. When lookup fails, the detection remains named and visible, its macro values and nutrition source are unavailable, and the response marks `nutrition_not_found`. A successful lookup still does not establish equivalence with the photographed recipe.

The macro total displayed to the user is computed from returned detections and gram-based portions:

$$\mathbf{t}_{\text{known}} = \sum_{q: M(\hat{c}_q) \neq \emptyset} s_q \mathbf{m}_q, \quad (15)$$

This is a known subtotal, with a separate incomplete flag whenever an included item lacks nutrition. The frontend uses $s_q = w_q/100$ for estimated grams and zero for excluded detections. It does not treat unknown values as zero or infer consumption from an unsaved scan.

The frontend is implemented as a tool-first interface. It supports image upload, visual overlays, nutrition cards, serving adjustment, macro totals, and recommendation review. This design keeps the result inspectable: the user sees both the model output and the nutrition values attached to that output.

C. Failure Modes Addressed by the Architecture

The layered design makes three failure modes inspectable: label drift, erroneous generated advice and missing nutrition. Precomputed mappings expose label-to-row choices, but do not certify those choices. Advice receives per-100-g database values and instructions against inventing nutrition measurements, yet generated text can still be wrong. Failed lookup remains explicit; successful low-threshold fallback is not automatically safe and requires review.

TABLE II
ALGORITHMIC SUMMARY OF END-TO-END INFERENCE.

Input: uploaded image x , detector f_θ , mapping M , nutrition database $\mathcal{N}$, serving vector s
Output: detections, mapped nutrition rows, meal macro vector, recommendation context
1. Validate image type, size, and non-empty payload.
2. Run $f_\theta(x)$ to obtain $\{(\hat{c}_q, \hat{\mathbf{b}}_q, \hat{p}_q)\}_{q=1}^{Q_x}$.
3. For each detection, query $M(\hat{c}_q)$.
4. If $M(\hat{c}_q) = \emptyset$, return explicit missing-nutrition state for that detection.
5. If a mapped row exists, attach display name, source, URL when present, and macro vector.
6. Return analysis JSON; the frontend computes known gram-scaled subtotals using Eq. (15).
7. Separately request advice with included food names, per-100-g values, goal and health context.
8. Validate the returned context before displaying advice; save a meal only after explicit user action.

V. DATASET ENGINEERING

A. Source Normalization

The dataset pipeline scans five YOLO-format source datasets. Table III lists the source-level image counts recorded before canonical merging. The sources contain overlapping and inconsistent label vocabularies; the raw union contains more than 100 label strings, but many strings refer to the same or closely related dish.

TABLE III
SOURCE DATASETS BEFORE CANONICAL MERGING.

Source	Description	Listed classes	Train	Valid	Test
S1	Indian food detection v1i	17	7,547	1,053	453
S2	Indian food v6i	29	8,088	213	105
S3	Indian food v2, seven-class subset	7	1,698	159	92
S4	IndianFoodNet YOLO export	30	11,387	1,073	576
S5	South Indian food detection v19i	31	6,478	1,294	581
Total	five YOLO-format sources	114+ raw labels	35,198	3,792	1,807

Each label row follows the normalized object detection format:

$$r_i = (c_i, x_i, y_i, w_i, h_i), \quad (16)$$

where c_i is the class index and (x_i, y_i, w_i, h_i) are normalized box coordinates. Source class indices are first decoded using each dataset’s class list. Raw class names are then converted into canonical labels through lowercasing, punctuation normalization, synonym rules, regional-name handling, and manual mappings.

The canonicalization process applies the following ordered transformation:

$$\phi_s(y) = \mu(\rho(\kappa(y))), \quad (17)$$

where $\kappa(\cdot)$ performs case and punctuation normalization, $\rho(\cdot)$ applies regional-name and spelling rules, and $\mu(\cdot)$ applies the manually verified semantic mapping table. The order matters. String cleanup alone can align `Al00Gobi` and `al00\_gobi`, but it cannot safely decide whether

sambhar, sambar, and sambar_satham should become the same class. Those cases require semantic review.

This step is necessary because food-name variation is semantic, not cosmetic. Without canonicalization, the same dish can be split across several classes. That would reduce training examples per class, confuse evaluation, and later break nutrition lookup. Table IV gives examples.

TABLE IV
EXAMPLES OF RAW LABEL NORMALIZATION.

Raw label variants	Canonical label
AlooGobi, aloo gobhi, aloo_gobi	aloo_gobi
CoconutChutney, thengai chutney	coconut_chutney
WhiteRice, satham, rice	rice
medu vadai, medu vada	medu_vada
lemon satham, lemon rice	lemon_rice

B. Split Construction

The export contains 72 classes and 36,852 train, 5,922 validation and 5,919 test image-label pairs (48,693 total). Additional augmentation was applied only to training, but downloaded sources already contained augmented variants. The September audit confirmed cross-partition duplicates; validation/test originals are not established as independent by the present builder.

Let $\mathcal{D}_c$ be the set of images containing class c . The split procedure aims to preserve class presence across train, validation, and test partitions:

$$\mathcal{D} = \mathcal{D}_{train} \cup \mathcal{D}_{val} \cup \mathcal{D}_{test}, \quad \mathcal{D}_{train} \cap \mathcal{D}_{val} \cap \mathcal{D}_{test} = \emptyset. \quad (18)$$

For each class c , the desired split proportions are approximately

$$\left[\frac{|\mathcal{D}_{train} \cap \mathcal{D}_c|}{|\mathcal{D}_c|}, \frac{|\mathcal{D}_{val} \cap \mathcal{D}_c|}{|\mathcal{D}_c|}, \frac{|\mathcal{D}_{test} \cap \mathcal{D}_c|}{|\mathcal{D}_c|} \right] \approx [0.70, 0.15, 0.15]. \quad (19)$$

Because an image can contain multiple food items, the split is a multi-label splitting problem rather than a simple class-balanced sampling problem.

TABLE V
GENERATED 72-CLASS DATASET SUMMARY.

Property	Value
Canonical classes	72
Train image-label pairs	36,852
Validation image-label pairs	5,922
Test image-label pairs	5,919
Total exported pairs	48,693

The dataset covers rice dishes, flatbreads, curries, gravies, snacks, chutneys, desserts, beverages, and non-vegetarian dishes. Representative examples include aloo gobi, dosa, fish curry, khandvi, palak paneer, and vegetable biryani.

TABLE VI
ALGORITHMIC SUMMARY OF CANONICAL DATASET CONSTRUCTION.

Input: source datasets $\{\mathcal{D}^{(s)}\}$, source class lists, canonicalization functions $\{\phi_s\}$
Output: canonical YOLO dataset with train, validation, and test splits
1. Read source <code>data.yaml</code> files and decode source class indices.
2. For each annotation row, convert source class $y^{(s)}$ into canonical class $\phi_s(y^{(s)})$.
3. Reject or flag annotations with invalid class indices or invalid normalized box coordinates.
4. Copy image-label pairs into a canonical export namespace.
5. Build multi-label image incidence matrix over the 72 canonical classes.
6. Split images into train, validation, and test partitions while preserving class coverage.
7. Apply augmentation only to the training split.
8. Generate class counts, verification samples, and contact sheets for audit.

C. Visual Verification

File counts do not prove that labels match visible foods. A visual verification script sampled images per class, parsed corresponding labels, extracted object crops, and generated contact sheets. Fig. 2 shows an example sheet. The check confirmed that all 72 classes have visible image samples and that the class order is aligned.

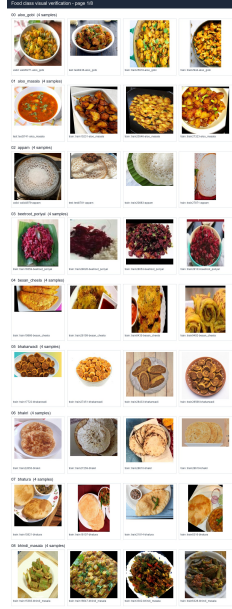


Fig. 2. Example class verification sheet generated from exported dataset images.

The verification found three sample-level issues: one `jalebi` image that does not visually resemble `jalebi`, one `kaara_chutney` annotation placed on a `vada`, and one `nandu_masala` box with extremely thin dimensions. These are annotation-cleaning issues rather than class-list failures.

For reviewer interpretation, these issues are important but bounded. They show that the verification step is capable of finding annotation-level defects, while the overall class-list validation remains intact. The current manuscript therefore treats the dataset as usable for system evaluation but not as a final public benchmark. A final dataset release should include complete annotation audit passes, duplicate-image detection, and class-wise error notes.

VI. DETECTOR TRAINING AND EVALUATION

A. Detection Objective

YOLO-style object detection optimizes localization, classification, and bounding-box distribution terms jointly. For a training batch $\mathcal{B}$, the objective can be represented as

$$\mathcal{L}(\theta) = \lambda_{box}\mathcal{L}_{box}(\theta; \mathcal{B}) + \lambda_{cls}\mathcal{L}_{cls}(\theta; \mathcal{B}) + \lambda_{dfl}\mathcal{L}_{dfl}(\theta; \mathcal{B}), \quad (20)$$

where $\mathcal{L}_{box}$ penalizes bounding-box localization error, $\mathcal{L}_{cls}$ penalizes class prediction error, and $\mathcal{L}_{dfl}$ is the distribution focal loss term used by the training implementation. The training configuration recorded in `args.yaml` uses $\lambda_{box} = 7.5$, $\lambda_{cls} = 0.5$, and $\lambda_{dfl} = 1.5$.

For a predicted box $\hat{\mathbf{b}}$ and ground-truth box $\mathbf{b}$, intersection-over-union is

$$\text{IoU}(\hat{\mathbf{b}}, \mathbf{b}) = \frac{\text{area}(\hat{\mathbf{b}} \cap \mathbf{b})}{\text{area}(\hat{\mathbf{b}} \cup \mathbf{b})}. \quad (21)$$

Predictions are counted as true positives when the predicted class is correct and the IoU exceeds the evaluation threshold. Precision and recall are then

$$P = \frac{TP}{TP + FP}, \quad R = \frac{TP}{TP + FN}. \quad (22)$$

Average precision for a class is the area under its precision-recall curve:

$$AP_c = \int_0^1 P_c(R) dR. \quad (23)$$

The reported mAP@50 is the mean AP over classes at IoU threshold 0.50, while mAP@50:95 averages AP over thresholds from 0.50 to 0.95.

B. Training Protocol

The detector was trained using YOLO11s with 640-pixel images, batch size 16, automatic mixed precision, and checkpoint saving. Training resumed from epoch 56 and continued to epoch 100 on a Tesla T4 GPU with 15.64 GB VRAM. The resumed stage took 10.636 hours. The final notebook exported `best.pt`, `last.pt`, intermediate checkpoints, `results.csv`, training curves, confusion matrices, and prediction batches.

Table VII summarizes the fixed training and evaluation protocol. The split, checkpoint rule, hardware, and metrics are reported before the aggregate results.

TABLE VII
TRAINING AND EVALUATION PROTOCOL.

Item	Setting
Detector	YOLO11s
Input image size	640 pixels
Batch size	16
Training schedule	resumed epoch 56 to epoch 100
Hardware	NVIDIA Tesla T4, 15.64 GB VRAM
Model selection	exported <code>best.pt</code> checkpoint
Validation split	5,922 images, 8,812 instances
Archived test split	5,919 images, 8,875 instances
Primary metrics	precision, recall, mAP@50, mAP@50:95

C. Detector Results

Fig. 3 shows the final training curves. The best validation mAP@50 recorded in the training log was 0.83379 at epoch 78. The final epoch recorded precision 0.8528, recall 0.8042, mAP@50 0.82806, and mAP@50:95 0.69164.

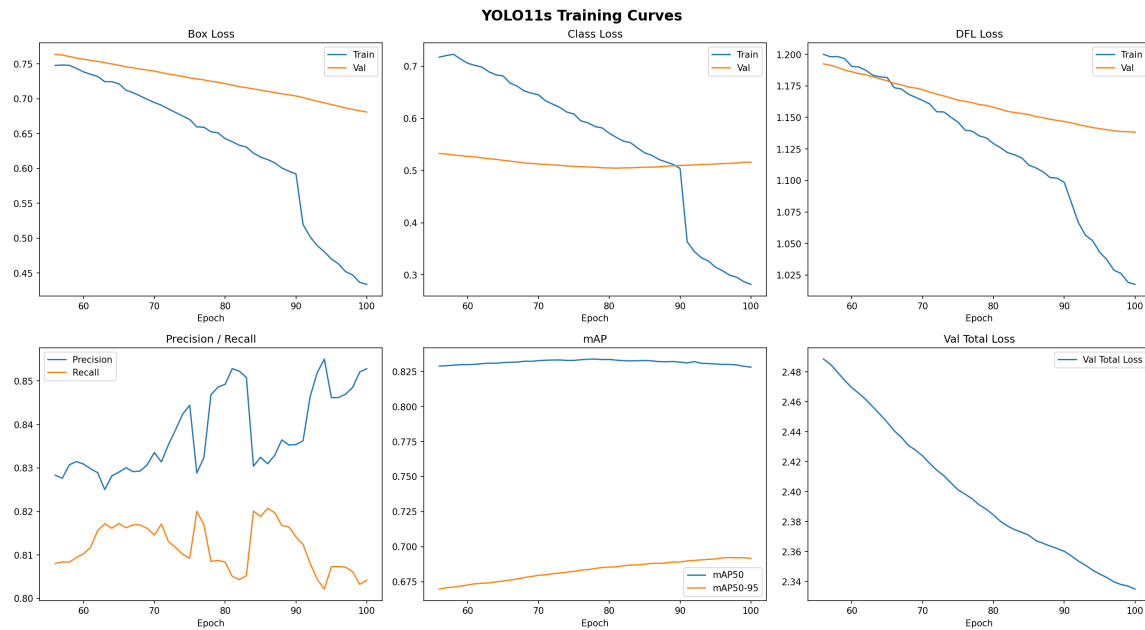


Fig. 3. YOLO11s training curves from the final resumed training stage.

The archived model evaluations were run on the partitions named validation and test. Table VIII retains the historical aggregate results. Their small numerical difference cannot establish generalization because the September audit found split overlap; independent evaluation is pending.

TABLE VIII
FINAL YOLO11s DETECTOR METRICS.

Split	Precision	Recall	mAP@50	mAP@50:95
Validation	0.847	0.807	0.830	0.692
Test	0.842	0.801	0.820	0.680

TABLE IX
CHECKPOINT-LEVEL EVIDENCE FROM THE FINAL TRAINING LOG.

Checkpoint	Precision	Recall	mAP@50	mAP@50:95
Epoch 78, best logged mAP@50	0.84685	0.80852	0.83379	0.68440
Epoch 99	0.85207	0.80325	0.82865	0.69212
Epoch 100	0.85280	0.80420	0.82806	0.69164

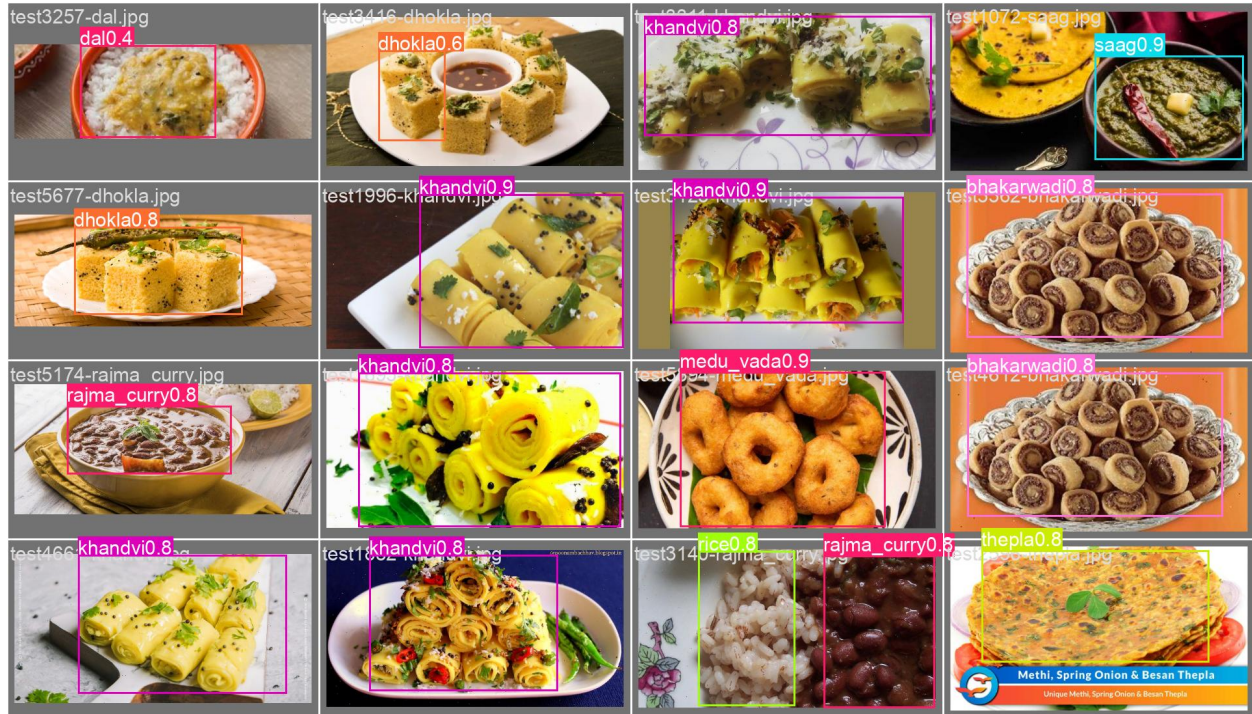


Fig. 4. Sample predictions from the archived test split.

The validation evaluation used 5,922 images and 8,812 instances. The archived test evaluation used 5,919 images and 8,875 instances. In the Kaggle Tesla T4 environment, the exported model

reported approximately 11.0 ms inference time per image during validation and test evaluation, excluding application upload, API serialization, nutrition lookup, and frontend rendering.

D. Benchmarking Interpretation

For a Transactions review draft, the current detector result should be interpreted as the completed primary benchmark rather than the complete benchmarking package. The repository contains an older `yolov8n\_indian.pt` model and the final YOLO11s run artifacts, but a fair detector-family comparison requires retraining or evaluating each candidate on the same 72-class split and with the same metrics. Table X states the benchmark matrix that should accompany a final journal submission. Rows marked completed are supported by the present artifacts; rows marked required should be run before external submission if the target journal expects model-family proof.

TABLE X
BENCHMARK MATRIX FOR JOURNAL REVIEW READINESS.

Benchmark	Status	Purpose
YOLO11s on 72-class split	completed	primary detector result reported in Table VIII
Earlier deployed model on same split	not comparable yet	requires aligned 72-class class list or restricted common-class evaluation
YOLOv8n or YOLO11n retrain	required before submission	establishes lightweight-model baseline
Larger YOLO variant retrain	required if compute allows	tests whether errors are capacity-limited
No-augmentation detector	required if compute allows	isolates augmentation benefit

The manuscript does not fabricate missing benchmark numbers. Instead, it reports the completed YOLO11s validation and archived test metrics and identifies the exact additional benchmark rows needed for a stronger IEEE Transactions submission.

VII. NUTRITION DATABASE AND MAPPING

A. Database Construction

The database imports the authors’ Indian Nutrient Databank workbook [18], [21]. The local file is byte-identical to the pinned public source; `research/evidence/nutrition-provenance.json` records its hash, revision and publication. This verifies identity, not downstream permissions or nutrient correctness. Offline preparation normalizes names and imports macros into SQLite. The 1,010 records include five supplemental entries.

Each database row is represented as

$$n_\ell = (\text{name}_\ell, \text{norm}_\ell, \text{kcal}_\ell, \text{protein}_\ell, \text{carb}_\ell, \text{fat}_\ell, \text{source}_\ell, \text{url}_\ell). \quad (24)$$

The normalized name is indexed because most lookups compare the normalized detector label against normalized database names. Complete macro fields are required before a row is retained; rows with missing calories, protein, carbohydrate, or fat values are excluded from the database import.

Five detector classes were absent from the available INDB sheet: `bhakarwadi`, `ghevar`, `jalebi`, `khandvi`, and `nandu_masala`. These classes were handled through source-backed supplemental rows rather than unsafe fuzzy matching [22], [23], [24], [25], [26]. Source metadata is stored so the application can distinguish native INDB entries from supplemental rows.

TABLE XI
SUPPLEMENTAL NUTRITION ROWS ADDED WHEN NO ACCEPTED INDB ROW WAS AVAILABLE.

Detector class	Source type	Mapping role
<code>bhakarwadi</code>	FatSecret	regional snack entry
<code>ghevar</code>	Clearcals	dessert entry
<code>jalebi</code>	Clearcals	dessert entry
<code>khandvi</code>	Clearcals	snack entry
<code>nandu_masala</code>	Clearcals	crab curry proxy

B. Mapping Policy

The mapping layer converts detector classes to database rows. The following rule summarizes offline database construction, not the more permissive runtime fallback:

$$M(c) = \begin{cases} M_{\text{manual}}(c), & \text{if precomputed manually,} \\ M_{\text{exact}}(c), & \text{if normalized names match,} \\ M_{\text{sup}}(c), & \text{if a supplemental row exists,} \\ M_{\text{fuzzy}}(c), & \text{if } \text{score}(c) \geq \tau, \\ \emptyset, & \text{otherwise.} \end{cases} \quad (25)$$

Here $\tau = 0.78$ is the offline builder’s threshold. Runtime first uses precomputed mappings, then normalized exact matching, containment with a length ratio of at least 0.5, variation containment without that score threshold, and a last-resort fuzzy score of at least 0.4. These existing rules are preserved, not certified as conservative enough. The runtime may abstain, but a returned row can still be inappropriate. All automatic matches display a review caution.

Let $\psi(\cdot)$ be the food-name normalizer and let $\sigma(a, b)$ denote the fuzzy similarity score between strings a and b . The fuzzy fallback can be written as

$$M_{\text{fuzzy}}(c) = \arg \max_{n_\ell \in \mathcal{N}} \sigma(\psi(c), \psi(n_\ell)). \quad (26)$$

The candidate is accepted only when the maximum score is at least τ . Manual mappings override this fallback because the highest string score is not always the safest nutrition match. For example, a regional spelling variant may have a low string score but a clear semantic target, while a visually related dish may have a high string score but a different macro profile.

Table XII shows representative mappings. Exact and manual mappings receive high confidence.

Approximate mappings, such as `ven_pongal` to plain khitchdi, are recorded with lower scores so they remain inspectable.

TABLE XII
REPRESENTATIVE DETECTOR-TO-NUTRITION MAPPINGS.

Detector class	Database food name	Score
<code>aloo_gobi</code>	Potato cauliflower (Aloo gobi)	1.00
<code>palak_paneer</code>	Spinach paneer (Palak paneer)	1.00
<code>rice</code>	Boiled rice (Uble chawal)	1.00
<code>chicken_biryani</code>	Chicken pulao	0.80
<code>ven_pongal</code>	Plain khitchdi	0.75

The final mapping table contains 103 entries because it includes both the 72 expanded dataset classes and legacy labels from an earlier deployed model. For the expanded class set, mapping coverage is 72 of 72 classes, or 100.00%.

C. Macro Consistency Verification

Macro verification was performed after mapping generation. For each mapped class, the script reads the mapped food row and computes the macro-derived energy:

$$e_{\text{macro}} = 4p + 4h + 9g, \quad (27)$$

where p , h , and g are protein, carbohydrate, and fat in grams. The difference

$$\Delta e = e_{\text{db}} - e_{\text{macro}} \quad (28)$$

is reported to expose rows where the database calorie value and macro-derived calorie estimate differ substantially. Small differences are expected because nutrition databases may include fiber conventions, rounding, recipe variation, and measurement uncertainty.

TABLE XIII
MACRO VERIFICATION SUMMARY FOR THE 72 EXPANDED DETECTOR CLASSES.

Verification category	Count
Heuristic mapping score at least 0.9	51
Review mappings with complete macro fields	21
Missing mappings	0
Rows missing required macro fields	0

The 21 review mappings are available rows with heuristic approximation flags, not missing data. All 72 mappings require independent assessment, including those scored at least 0.9. No category in this table is a clinical-grade reference or an independently measured mapping accuracy.

TABLE XIV
ALGORITHMIC SUMMARY OF NUTRITION DATABASE BUILD AND MAPPING.

Input: INDB spreadsheet, supplemental rows, canonical class list $\mathcal{C}$, legacy class list
Output: SQLite database with <code>indb_foods</code> and <code>yolo_mappings</code>
1. Load required columns: food name, energy, protein, carbohydrate, and fat.
2. Normalize food names and remove duplicate normalized rows.
3. Drop rows with incomplete macro fields.
4. Append source-attributed supplemental rows for uncovered classes.
5. Create indexed <code>indb_foods</code> table with source metadata.
6. Load canonical detector classes and legacy deployed labels.
7. Resolve each class using manual mapping, exact normalized match, supplemental row, and thresholded fuzzy fallback.
8. Insert accepted mappings and leave unsafe classes unmapped.
9. Run macro verification and write class-level audit reports.

VIII. PROTOTYPE IMPLEMENTATION

The runtime backend uses FastAPI, PyTorch, Ultralytics YOLO, Pillow, SQLite and AioSQLite; Pandas and OpenPyXL belong to offline database preparation [27], [28], [14]. The September frontend uses Next.js 16, React, TypeScript, Tailwind CSS and Lucide icons [29]. Fig. 5 preserves a historical frontend example.



Fig. 5. Frontend output showing detected food items with bounding boxes and nutrition-linked cards.

The analysis endpoint validates nonempty uploads, supported decoding, a 4 MiB server file ceiling, at most 25 million decoded pixels, request-rate limits and inference capacity. The frontend normalizes orientation, resizes to at most 1,600 pixels and compresses below 3.8 MiB; the same processed image is displayed so coordinates align. The API returns labels, confidence, boxes, dimensions, per-100-g macros and provenance. The client allows 120 seconds for analysis and

distinguishes empty detection from invalid upload, busy, timeout and server errors. Users can exclude detections, estimate grams and explicitly save a meal. Only saved meals affect daily totals.

The recommendation request contains included food names, per-100-g database values, goal and health context. Production uses Groq followed by local fallback; no paid alternative is configured. The asynchronous client has a 20-second timeout with automatic retries disabled. At most two provider calls run per instance with no waiting queue; saturation returns retryable 503. Source and returned context are validated separately. Changing context immediately hides stale advice, and late responses are discarded. The badge describes response context, not medical suitability. Advice quality has not been independently evaluated.

A. Macro Target Engine

The deterministic macro engine converts a user goal and daily calorie target into carbohydrate, protein, and fat gram targets. Let

$$\mathbf{g} = [g_c, g_p, g_f]^\top \quad (29)$$

be the base macro percentage vector for a dietary goal, where g_c , g_p , and g_f are carbohydrate, protein, and fat percentages. Let

$$\mathbf{r} = [r_c, r_p, r_f]^\top \quad (30)$$

be the health-condition modifier vector. The adjusted macro percentages are computed by normalized elementwise scaling:

$$\tilde{\mathbf{g}} = 100 \cdot \frac{\mathbf{g} \odot \mathbf{r}}{\mathbf{1}^\top (\mathbf{g} \odot \mathbf{r})}, \quad (31)$$

where $\odot$ denotes elementwise multiplication. Given a calorie target E , the gram targets are

$$T_c = \frac{E \tilde{g}_c}{400}, \quad T_p = \frac{E \tilde{g}_p}{400}, \quad T_f = \frac{E \tilde{g}_f}{900}. \quad (32)$$

The denominators encode 4 kcal/g for carbohydrate, 4 kcal/g for protein, and 9 kcal/g for fat.

TABLE XV
BASE MACRO SPLITS IMPLEMENTED IN THE RECOMMENDATION SERVICE.

Goal	Carbohydrate	Protein	Fat
Weight loss	40%	35%	25%
Muscle gain	40%	30%	30%
Maintenance	50%	25%	25%
Endurance	55%	20%	20%

TABLE XVI
HEALTH-CONDITION MODIFIERS IMPLEMENTED IN THE MACRO ENGINE.

Condition	Carb modifier	Protein modifier	Fat modifier
None	1.00	1.00	1.00
Diabetic	0.80	1.15	1.00
Hypertension	1.00	1.00	0.90
Heart disease	1.05	1.00	0.80
High cholesterol	1.05	1.10	0.75
Digestive issues	1.00	0.90	0.85
Kidney disease	1.15	0.60	1.00
Anemia	1.00	1.10	0.90
Thyroid disorder	1.00	1.05	1.00

B. Recommendation Generation Guardrail

The prompt uses the included food records $\mathcal{F}_x^{inc}$, their database values per 100 g, goal identifier g and session-only health-context identifier h :

$$\mathcal{Q} = (\mathcal{F}_x^{inc}, g, h). \quad (33)$$

Neither target calories, macro targets, selected grams, consumed totals nor journal history are transmitted in this request. The prompt describes its values as database estimates per 100 g, not measured intake. Advice text does not modify the frontend’s deterministic nutrition arithmetic. This is an implementation boundary, not a guarantee against incorrect advice.

IX. VALIDATION RESULTS

Table XVII summarizes the system-level validation. These results cover detector evaluation, dataset integrity, mapping coverage, database construction, macro logic, and application workflow

checks.

TABLE XVII
END-TO-END VALIDATION OUTCOMES.

Validation item	Result
Canonical detector classes	72
Exported image-label pairs	48,693
Train / validation / test images	36,852 / 5,922 / 5,919
Classes with visual samples	72 / 72
Validation mAP@50 / mAP@50:95	0.830 / 0.692
Test mAP@50 / mAP@50:95	0.820 / 0.680
Validation / test mAP@75	0.765 / 0.751
Nutrition records	1,010
YOLO-to-database mappings	103
Expanded-class nutrition coverage	72 / 72
Macro health profiles	9

Service-level lookup tests were run for classes that previously suffered from incorrect fuzzy matches. Inputs such as `aloo_gobi`, `palak_paneer`, `rice`, `AlooGobi`, and `WhiteRice` resolved to the intended database display names. Backend validation also covered non-image uploads, empty files, maximum-size rejection, unavailable service states, and no-food responses. Frontend workflow validation confirmed image preview, overlay alignment, nutrition-card rendering, gram-portion updates, remaining macro display, and recommendation refresh. These checks matter because ordinary detector metrics would not catch a nutrition mapping regression, API data-contract mismatch, or stale frontend recommendation state.

X. ABLATION AND BENCHMARK EVIDENCE

A. Why Detector Metrics Alone Are Insufficient

The archived detector metrics describe performance on the old export but cannot establish independent generalization after the split-overlap findings. They do not measure nutrition accuracy either. For a detection q , the user-visible estimate depends on the detector and selected database row:

$$\mathbf{m}_q = \mathbf{m}_{M(\hat{c}_q)}. \quad (34)$$

If $\hat{c}_q$ is correct but $M(\hat{c}_q)$ is wrong, then the nutrition result is wrong. If $\hat{c}_q$ is wrong but maps to a plausible-looking food row, the frontend can display a confident but incorrect macro card. The system must therefore be evaluated at multiple levels.

TABLE XVIII
EVALUATION LEVELS REQUIRED FOR THE COMPLETE SYSTEM CLAIM.

Level	Metric or check	Evidence in this manuscript
Dataset	class coverage, visual sample verification	72/72 classes visually sampled
Detector	precision, recall, mAP@50, mAP@50:95	validation and archived test metrics
Mapping	coverage and failed mappings	72/72 coverage, 0 failed mappings
Nutrition	complete macros and energy consistency	0 failed macro rows
Application	API/frontend data contract	prototype and lookup validation

B. Nutrition-Layer Ablation

The nutrition-layer ablation compares the full conservative mapping system with simplified variants. Let

$$\text{Cov}(M) = \frac{|\{c \in \mathcal{C} : M(c) \neq \emptyset\}|}{|\mathcal{C}|}. \quad (35)$$

For the expanded dataset, the implemented mapping has $\text{Cov}(M) = 72/72 = 1.0$.

TABLE XIX
NUTRITION MAPPING ABLATION EVIDENCE.

Variant	Coverage outcome	Interpretation
Full mapping with manual, exact, supplemental, and thresholded fuzzy rules	72/72 classes, 0 failed mappings	completed system result
Supplemental rows removed	at most 67/72 guaranteed classes	five classes are absent from the available INDB sheet
Precomputed mapping table removed	fallback lookup remains; correctness effect unmeasured	runtime matching depends on name matching; independent comparison is pending
Macro verification removed	coverage may appear complete, but failed macro fields would be hidden	verification is needed to prove complete per-100g macro availability

This ablation demonstrates why the nutrition layer is not an implementation detail. The five supplemental rows are necessary for full expanded-class coverage, while the precomputed mappings prevent runtime fuzzy matching from becoming the primary source of nutritional truth.

C. Detector Benchmark and Ablation Requirements

The completed detector benchmark is the YOLO11s run reported in Section VI. For a stronger Transactions submission, the detector benchmarking should be extended to a controlled family comparison:

$$\Delta_{\text{model}} = \text{mAP}_{50:95}(f_{\theta_a}) - \text{mAP}_{50:95}(f_{\theta_b}), \quad (36)$$

where f_{θ_a} and f_{θ_b} are trained or evaluated under the same 72-class split. Similarly, an augmentation ablation should report

$$\Delta_{\text{aug}} = \text{mAP}_{50:95}(f_{\theta}^{\text{aug}}) - \text{mAP}_{50:95}(f_{\theta}^{\text{noaug}}). \quad (37)$$

The current repository preserves older model files, but the manuscript does not treat them as fair baselines because their class lists and training corpora are not guaranteed to match the final 72-class export. The correct journal action is to run the benchmark matrix in Table X before external submission, then replace the “required” rows with measured values.

D. Reviewer-Ready Proof Checklist

Table XX summarizes what is already supported and what remains to satisfy a rigorous IEEE Transactions review.

TABLE XX
PROOF STATUS FOR REVIEW READINESS.

Evidence item	Status	Notes
Canonical dataset lifecycle	complete	source counts, normalization, split, verification reported
Held-out detector test	complete	test mAP@50 and mAP@50:95 reported
Nutrition coverage proof	complete	72/72 mappings and 0 failed mappings
Macro field/heuristic checks	complete	51 higher-score, 21 review; independent correctness unmeasured
Detector-family benchmark	incomplete	needs same-split YOLOv8/YOLO11 variant runs
Augmentation ablation	incomplete	needs no-augmentation retrain or controlled comparison
Portion-estimation benchmark	not implemented	outside current prototype scope

XI. DISCUSSION

The results support the central design decision: the detector and nutrition layer must be evaluated together. The final YOLO11s detector provides usable object detection performance on the expanded 72-class dataset. However, detector output becomes useful to a diet application only after labels are mapped to reliable food-composition rows. A wrong mapping can turn a correct visual detection into misleading nutrition feedback.

The mapping design attempts to make risk inspectable. Developer-selected entries handle name variants; supplements address coverage gaps; the interface exposes sources and approximate-match cautions. No independent study establishes reduced wrong matches or clinical risk. The exact/automatic/precomputed comparison measures availability only. The reviewer template and abstention-aware scorer support future correctness evaluation without treating current mappings as truth.

The implemented full-stack workflow also changes the interpretation of the result. The project is not only a detector benchmark; it is a traceable image-to-recommendation pipeline. The frontend exposes labels, boxes, confidence scores, macro cards, serving controls, remaining daily macro values, and recommendation text. This makes incorrect detections and uncertain serving assumptions easier to notice than in a black-box calorie-estimation interface.

The system also shows the value of simple validation artifacts. Contact sheets found annotation problems that file counts would miss. Runtime lookup tests caught mapping behavior that detector mAP would not measure. API and frontend workflow checks caught data-contract issues that model evaluation would not expose. This suggests that nutrition-aware food recognition should report model metrics, nutrition provenance, and application-level validation together.

XII. ETHICAL AND PRACTICAL CONSIDERATIONS

The prototype is intended for dietary logging assistance and nutrition awareness, not for diagnosis, treatment, or clinical decision-making. The system displays nutrition values from database rows and uses generated text only after backend macro values have already been calculated. This separation reduces the risk that a language model will invent calories, protein, carbohydrates, or fat values.

The public demo processes photos on the backend without deliberately retaining them. Its version-1 journal stores saved meals and goal preferences only in the browser, without images or health selections. Groq receives included food names, per-100-g values, goal and context, but not photos, gram weights or saved history. Provider processing remains subject to third-party policies, not a zero-retention guarantee. Storage failures leave scanning usable and are surfaced to users. Clinical rules and generated advice have not been validated.

XIII. LIMITATIONS

Experimental independence is the strongest limitation: cross-split duplicates and source augmentation families qualify all archived results. A candidate split is not a newly trained model or independent test. Filename ancestry cannot resolve every crop, transformed copy or annotation defect. Nutrients are database estimates scaled by user-estimated grams, not measurements

of ingredients or mass. All mappings require independent review, including 21 lower-scoring approximations. Advice and health-target correctness have not been clinically assessed. Source rights and author consent remain prerequisites.

The recommendation layer is also not clinically validated. Although macro arithmetic is deterministic and testable, natural-language advice depends on provider behavior when an external language model is configured. The prototype should therefore be interpreted as a decision-support system rather than a clinical nutrition instrument. It can reduce logging effort and provide useful macro context, but it should not be presented as medical advice.

XIV. FUTURE WORK

Next work is evidence-focused: review the grouped manifest, annotation conflicts and upstream augmented variants; freeze a defensible protocol; use appropriate retraining or untouched external data; and compare mappings with independent references. A fair detector baseline needs the same split, input size and training budget. Advice-quality claims need qualified assessment. Browser-only meal history already exists; accounts, cloud storage, portion estimation and medical personalization are not prerequisites.

XV. CONCLUSION

This work implements *AI-Based Food Recognition with Nutrition-Aware Recommendations* as an educational web application and audits the evidence behind its detector and nutrition layer. The existing model and 72-class export remain unchanged. Exact cross-split duplication and unresolved source-family relationships prevent interpreting the archived detector metrics as independent generalization. The mapping layer provides lookup coverage for all 72 classes, but independent recipe-equivalence assessment is outstanding. Gram-based review, saved-only daily totals, explicit source details, and response-context labels improve inspectability; software tests do not establish clinical validity. The next research milestone is reviewed source grouping and annotation handling followed by a frozen evaluation protocol, appropriate retraining or untouched external testing, and independent mapping assessment. Asset permissions and author approval remain prerequisites for submission.

DATA AND CODE AVAILABILITY

The implementation, database-build scripts, configuration files, trained-model references, and generated evaluation artifacts are maintained in the project repository used for this study. Source datasets and supplemental nutrition rows should be accessed through their original providers according to their licenses and redistribution terms. Trained weights and local database exports can be made available for review subject to the redistribution permissions of the underlying datasets and nutrition sources.

ACKNOWLEDGMENT

The authors thank the Department of Computer Science and Engineering, National Institute of Technology Srinagar, for academic support during the final year project.

REFERENCES

- [1] project-z0tql, “Indian food detection, version 1,” <https://universe.roboflow.com/project-z0tql/indian-food-detection-kzw9g/dataset/1>, source/version recovered from local dataset export metadata, inspected 2026-09-12; image-level rights require separate review.
- [2] microplastics-vypxl, “Indian food, version 6,” <https://universe.roboflow.com/microplastics-vypxl/indian-food-txofy/dataset/6>, source/version recovered from local dataset export metadata, inspected 2026-09-12; image-level rights require separate review.
- [3] indianfood, “Indianfood-7, version 2,” https://universe.roboflow.com/indianfood/indian_food-pwzlc/dataset/2, source/version recovered from local dataset export metadata, inspected 2026-09-12; image-level rights require separate review.
- [4] IndianFoodNet contributors, “Indianfoodnet-30, version 1,” <https://universe.roboflow.com/indianfoodnet/indianfoodnet/dataset/1>, source/version recovered from local dataset export metadata, inspected 2026-09-12; image-level rights require separate review.
- [5] bharani-lucid, “South indian food detection, version 19,” <https://universe.roboflow.com/bharani-lucid/south-indian-food-detection/dataset/19>, source/version recovered from local dataset export metadata, inspected 2026-09-12; image-level rights require separate review.
- [6] W. Min, S. Jiang, L. Liu, Y. Rui, and R. Jain, “A survey on food computing,” *ACM Computing Surveys*, vol. 52, no. 5, pp. 1–36, 2019.
- [7] Y. Kawano and K. Yanai, “Foodcam: A real-time food recognition system on a smartphone,” *Multimedia Tools and Applications*, vol. 74, pp. 5263–5287, 2014.
- [8] A. Meyers, N. Johnston, V. Rathod, A. Korattikara, A. Gorban, N. Silberman, S. Guadarrama, G. Papandreou, J. Huang, and K. P. Murphy, “Im2calories: Towards an automated mobile vision food diary,” *Proceedings of the IEEE International Conference on Computer Vision*, pp. 1233–1241, 2015.

- [9] S. Ren, K. He, R. Girshick, and J. Sun, “Faster R-CNN: Towards real-time object detection with region proposal networks,” *Advances in Neural Information Processing Systems*, vol. 28, 2015.
- [10] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, “You only look once: Unified, real-time object detection,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2016, pp. 779–788.
- [11] J. Redmon and A. Farhadi, “YOLOv3: An incremental improvement,” *arXiv preprint arXiv:1804.02767*, 2018.
- [12] R. Agarwal, T. Choudhury, N. J. Ahuja, and T. Sarkar, “IndianFoodNet: Detecting indian food items using deep learning,” *International Journal of Computational Methods and Experimental Measurements*, vol. 11, no. 4, pp. 221–232, 2023.
- [13] M. Goel, S. Dargar, S. Ghatak, N. Verma, P. Chauhan, A. Gupta, N. Vishnumolakala, H. Amuru, E. Gambhir, R. Chhajer, M. Jain, A. Jain, S. Garg, N. Narwade, N. Verhwani, A. Tiwari, K. Vashishtha, and G. Bagler, “Dish detection in food platters: A framework for automated diet logging and nutrition management,” *arXiv preprint arXiv:2305.07552*, 2023.
- [14] Ultralytics, “Ultralytics yolo documentation,” 2024, software documentation; accessed during project development.
- [15] T.-Y. Lin, M. Maire, S. Belongie, J. Hays, P. Perona, D. Ramanan, P. Dollár, and C. L. Zitnick, “Microsoft COCO: Common objects in context,” in *European Conference on Computer Vision*. Springer, 2014, pp. 740–755.
- [16] O. Prabhu, “Khana: A comprehensive Indian cuisine dataset,” *arXiv preprint arXiv:2509.06006*, 2025.
- [17] Y. Arora, A. Arun, and C. V. Jawahar, “What is there in an Indian Thali?” in *Proceedings of the Indian Conference on Computer Vision, Graphics and Image Processing*, 2025.
- [18] A. Vijayakumar, H. B. Dubasi, A. Awasthi, and L. M. Jaacks, “Development of an Indian food composition database,” *Current Developments in Nutrition*, vol. 8, no. 7, p. 103790, 2024.
- [19] S. K. Gupta, L. Dey, P. P. Das, G. Trilok-Kumar, and R. Jain, “Enhancing FKG.in: Automating Indian food composition analysis,” *arXiv preprint arXiv:2412.05248*, 2024.
- [20] C. O’Hara, G. Kent, A. C. Flynn, E. R. Gibney, and C. M. Timon, “An evaluation of ChatGPT for nutrient content estimation from meal photographs,” *Nutrients*, vol. 17, no. 4, p. 607, 2025.
- [21] A. Vijayakumar, H. B. Dubasi, A. Awasthi, and L. M. Jaacks, “Indian Nutrient Databank (INDB) workbook,” <https://github.com/lindsayjaacks/Indian-Nutrient-Databank-INDB/blob/fbdb62bec519c6e2468582a799fb47e194b42558/INDB.xlsx>, 2024, workbook modified November 18, 2024; accessed September 12, 2026. Local file byte identity verified; see repository provenance evidence.
- [22] FatSecret India, “Evolve bhakarwadi nutrition facts per 100 g,” <https://www.fatsecret.co.in/calories-nutrition/evolve/bhakarwadi/100g>, 2026, accessed 2026-06-06.
- [23] Clearcals, “Ghevar recipe calories and nutrition per 100 g,” <https://clearcals.com/recipes/ghevar/>, 2026, accessed 2026-06-06.
- [24] —, “Jalebi recipe calories and nutrition per 100 g,” <https://clearcals.com/recipes/jalebi/>, 2026, accessed 2026-06-06.
- [25] —, “Khandvi recipe calories and nutrition per 100 g,” <https://clearcals.com/recipes/khandvi/>, 2026, accessed 2026-06-06.
- [26] —, “Nandu kari recipe calories and nutrition per 100 g,” <https://clearcals.com/recipes/nandu-kari/>, 2026, accessed 2026-06-06.
- [27] FastAPI, “Fastapi documentation,” 2024, software documentation; accessed during implementation.
- [28] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga *et al.*, “PyTorch: An imperative style, high-performance deep learning library,” *Advances in Neural Information Processing Systems*, vol. 32, 2019.

[29] Vercel, “Next.js documentation,” 2024, software documentation; accessed during implementation.